

WAIT (Wait Execution)

Milenko Burlica

28 July, 2026

Table of Contents

1	Globales Feature: condition (Bedingte Ausführung).....	4
2	Engine-spezifische Parameter	5
3	Unterstützte Syntax-Beispiele	6
3.1	1. Implizite Milisekunden (Standard-Fallback)	6
3.2	2. Explizite Sekunden-Erkennung	6
4	 Konsolen-Logging & Live-Ausgabe (Log Alignment).....	7
4.1	Konsole-Ausgabe bei erfolgreichem Warten:.....	7
4.2	Konsole-Ausgabe bei einem Fehler (z. B. asynchroner Abbruch / InterruptedException):	7
5	 Wichtige YAML-Syntaxregeln & Sonderzeichen-Handling	8

Die Engine blockiert während der Wartezeit **ausschließlich den Thread des aktuellen Clients**. Der zugrundeliegende Server, Jenkins-Runner oder parallele Testfälle anderer Clients werden **nicht** beeinträchtigt. Der Betriebssystem-Scheduler gibt die CPU-Ressourcen während des Wartens automatisch für die anderen 99 parallelen Prozesse frei.

1 Globales Feature: `condition` (Bedingte Ausführung)

Mit dem optionalen Feld `condition` kann gesteuert werden, ob die Pause eingelegt werden soll.

Wenn die Bedingung nicht zutrifft (`false`), wird der Schritt übersprungen (`SKIPPED`) und der Test läuft fehlerfrei und ohne Zeitverlust weiter.

YAML

```
# Wartet nur, wenn der vorherige Schritt explizit signalisiert, dass ein Sync nötig ist
- type: WAIT
  condition: "'{B[needs_sync]}' == 'true'"
  value: "5s"
```

2 Engine-spezifische Parameter

Neben den Standardfeldern besitzt die Wait-Engine ein zentrales Steuerungs-Feld im YAML:

- `value` (**String/Number, Pflichtfeld**): Die Dauer der Pause.
 - Wird nur eine Zahl übergeben (z. B. `1000`), interpretiert die Engine dies standardmäßig als **Milisekunden**.
 - Die Engine verfügt über eine **intelligente Zeiteinheiten-Erkennung**: Endet der String auf `s` (und nicht auf `ms`), wird der Wert automatisch in **Sekunden** umgerechnet.
 - Ist das Feld leer oder `null`, wird aus Sicherheitsgründen standardmäßig ein Fallback auf `0 ms` angewendet.

3 Unterstützte Syntax-Beispiele

3.1 1. Implizite Milisekunden (Standard-Fallback)

Wird keine Einheit oder die explizite Endung `ms` angegeben, wartet das Framework die exakte Anzahl an Milisekunden.

YAML

```
- type: WAIT  
  value: "2500" # Wartet exakt 2500 Milisekunden (2,5 Sekunden)
```

3.2 2. Explizite Sekunden-Erkennung

Durch das Anhängen eines `s` (Groß-/Kleinschreibung wird ignoriert) konvertiert die Engine den Wert direkt in ein sekundenbasiertes Warten.

YAML

```
- type: WAIT  
  value: "5s" # Wird intern automatisch als 5000 ms verarbeitet
```

4 Konsolen-Logging & Live-Ausgabe (Log Alignment)

Passend zur `Cmd-Engine` und `Oc-Engine` verfügt auch die Wait-Engine über ein strikt vertikal ausgerichtetes Logging-System. Alle System-Tags (`>>> WAIT` , `>>> RESULT`) haben eine feste Breite von **14 Zeichen bis zum Doppelpunkt** und garantieren ein sauberes Schriftbild untereinander.

4.1 Konsole-Ausgabe bei erfolgreichem Warten:

Plaintext

```
>>> WAIT      : Pausing execution for 5000 ms (5s)
>>> RESULT    : SUCCESS (Done)
```

4.2 Konsole-Ausgabe bei einem Fehler (z. B. asynchroner Abbruch / `InterruptedException`):

Sollte der Thread während der Wartezeit extern unterbrochen oder gecancelt werden, greift das *Fail-Fast-Prinzip*: Die Engine fängt den Fehler ab, loggt das Fail-Resultat in Rot und bricht den Testlauf sofort ab.

Plaintext

```
>>> WAIT      : Pausing execution for 10000 ms (10s)
>>> RESULT    : FAILED (Wait interrupted)
```

5 💡 Wichtige YAML-Syntaxregeln & Sonderzeichen-Handling

1. **Schutz von Dynamic-Tags { }** im **value-Feld** Falls die Wartezeit dynamisch aus einer TestCase-Variable geladen werden soll, muss der Wert zwingend in Anführungszeichen gesetzt werden, um den YAML-Parser nicht zu blockieren.

YAML

Richtig: Lädt die Wartezeit dynamisch (z.B. "3s" oder "3000") aus den Variablen

```
- type: WAIT
  value: "{B[custom_timeout]}"
```